Week 14 – Security and Protection

Operating Systems Ch 16, 17

Part 1

The Security Problem

Security vs Protection

Security: A measure of confidence that the integrity of a system and its data will be preserved

Protection: A set of mechanisms that control the access of processes and users to resources

Aspect	Security	Protection
Focus	Protecting the system from external threats	Controlling internal resource access
Target	Attackers, malware, network threats	Processes, users, roles
Scope	Physical, network, OS, application	Domains, access matrix, privileges
Textbook	Ch 16	Ch 17

Three Principles of Security: CIA Triad

Principle	Full Name	Description	Violation Example
Confidentiality	**Confidentiality**	Only authorized users can access information	Data eavesdropping, information leakage
Integrity	**Integrity**	Only authorized users can modify information	Data tampering, source code manipulation
Availability	**Availability**	Authorized users can access resources when needed	DoS attacks, system outages

Additional types of security violations:

- **Theft of Service**: Unauthorized use of resources (e.g., cryptocurrency mining, spam relay)
- **Denial of Service**: Preventing legitimate users from using the system

Threat vs Attack

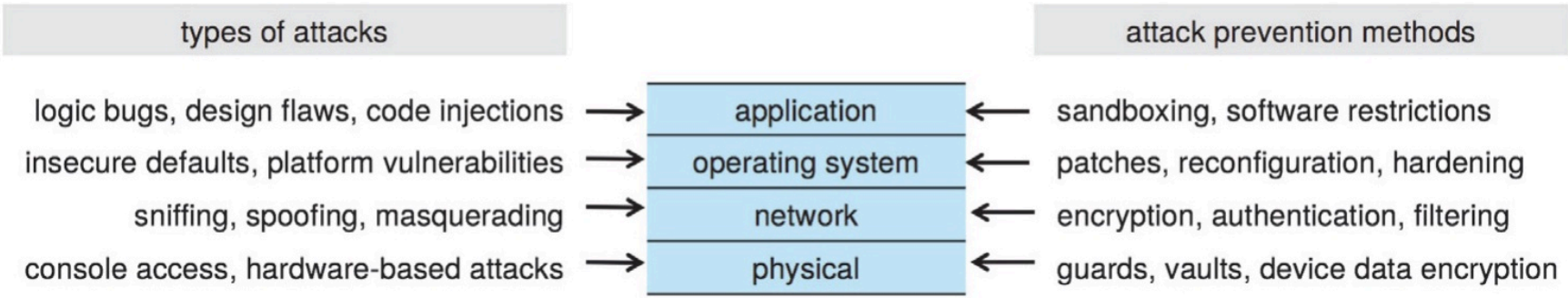
Threat: A potential security violation – the discovery of a vulnerability

Attack: An actual attempt to breach security – a concrete action

Major attack types:

Attack Type	Description
Masquerading	Impersonating another user/system
Replay Attack	Maliciously repeating a valid data transmission
Man-in-the-Middle	Intercepting or modifying data during communication
Session Hijacking	Taking over an active communication session

Four-Layer Model of Security



Silberschatz, Figure 16.1 – The four-layered model of security

- Security is only as strong as its **weakest link** (chain analogy)
- **Human Factor**: Attacks exploiting people, such as social engineering and phishing
- **Attack Surface**: The set of points where an attacker can attempt to infiltrate

Part 2

Program Threats

Malware Overview

Malware: Software designed to exploit, disable, or damage a system

Type	Description
Trojan Horse	Disguised as a legitimate program but executes malicious functions
Spyware	Secretly collects user information and sends it externally (adware, keylogger)
Ransomware	Encrypts files and demands payment for decryption
Back Door (Trap Door)	A secret access path left by a developer
Logic Bomb	Malicious code that activates when specific conditions are met

- Trojan Horse variant: **login emulator** (fake login screen)
- Violating the **principle of least privilege** maximizes malware damage

Trojan Horse Example

Login Emulator Attack Scenario:

```
1. Attacker leaves a fake login screen running
2. Victim attempts to log in → "Password error" message displayed
3. In reality, the ID/PW are sent to the attacker
4. The fake program exits and the real login prompt appears
5. Victim thinks "It was a typo" and logs in again (succeeds normally)
```

Defenses:

- Use non-trappable key sequences like **Ctrl+Alt+Delete** (Windows)
- Display session usage information at session termination
- Verify URL accuracy (phishing prevention)

Ransomware and the Principle of Least Privilege

Ransomware Operation:

```
1. Malware infection (phishing emails, vulnerable services, etc.)
2. Encrypts files on the system using strong encryption algorithms
3. Demands payment (cryptocurrency) in exchange for the decryption key
4. No guarantee of receiving the decryption key even after payment
```

The Principle of Least Privilege:

- Grant programs and users only the **minimum privileges** needed to perform their tasks
- Excessive privileges allow malware to take over the entire system
- The OS should provide **fine-grained access control**

Code Injection – Buffer Overflow

Most security threats occur through code injection

Buffer Overflow: Overwriting adjacent memory by providing input that exceeds buffer boundaries

```c
#include <stdio.h>
#define BUFFER_SIZE 64

void vulnerable(char *input) {
    char buffer[BUFFER_SIZE];
    strcpy(buffer, input);   // No length check → overflow possible
}

// Safe version
void safe(char *input) {
    char buffer[BUFFER_SIZE];
    strncpy(buffer, input, sizeof(buffer) - 1);  // Length limited
    buffer[sizeof(buffer) - 1] = '\0';
}
```

- `strcpy()`, `sprintf()`, `gets()` are **dangerous functions**
- Use size-aware functions like `strncpy()`, `snprintf()` instead

Consequences of Buffer Overflow

```
Stack (high address → low address):

┌──────────────────┐
│ Return Address   │  ← (3) If attacker overwrites this, they control code flow
├──────────────────┤
│ Saved EBP        │  ← (2) Frame pointer can be corrupted
├──────────────────┤
│ Local Variables  │  ← (2) Adjacent variables overwritten
├──────────────────┤
│ buffer[64]       │  ← (1) Within padding range: no effect
└──────────────────┘

      ↑ overflow direction
```

Consequences depending on overflow size:

1. **Small overflow** → Absorbed by padding area, no impact
2. **Medium overflow** → Overwrites adjacent variables, possible program crash
3. **Large overflow** → **Return address corruption** → Arbitrary code (shellcode) execution

Shellcode and NOP-sled

Shellcode: Code injected by the attacker (typically spawns a shell)

```
void exploit(void) {
    execvp("/bin/sh", "/bin/sh", NULL);  // Spawn shell
}
```

Attack process:

1. Attacker includes shellcode in user input
2. Buffer overflow changes the return address to point to the shellcode location
3. Shellcode executes when the function returns

NOP-sled: Since hitting the exact address is difficult, a large number of NOP instructions are placed before the shellcode

```
[NOP NOP NOP NOP NOP ... NOP] [SHELLCODE]
 ←── NOP-sled ──────────→     ← payload
```

- **Script kiddie**: A novice attacker who uses exploits created by other hackers

SQL Injection

An attack that inserts SQL statements into user input to manipulate the database

```
Normal input:     username = "alice"
SQL query:        SELECT * FROM users WHERE name = 'alice'

Malicious input:  username = "' OR '1'='1"
SQL query:        SELECT * FROM users WHERE name = '' OR '1'='1'
→ All records are returned (authentication bypass)
```

Defenses:

- Use **Prepared Statements** (parameterized queries)
- Input **validation** and **sanitization**
- Grant minimum privileges to DB accounts

```java
// Safe approach (Prepared Statement)
PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM users WHERE name = ?");
stmt.setString(1, username);  // Automatic escaping
```

Viruses

Virus: Self-replicating code that spreads by **inserting itself** into other programs

- **Requires a host program** (cannot run independently)
- Mainly spreads through user actions (executing files, email attachments)

Virus Type	Description
File Virus	Inserts into executable files (parasitic virus)
Boot Virus	Infects the boot sector, executes before OS loads
Macro Virus	Inserts into document macros (VBA, etc.), infects Office files
Polymorphic	Mutates code on each infection → evades signature detection
Encrypted	Contains encrypted virus + decryption code

Worms

Worm: Malware that propagates **autonomously** through networks

- **No host program required** (can run independently)
- Automatically spreads through the network → generates massive traffic

Virus vs Worm Comparison:

Property	Virus	Worm
Host Program	Required	Not required
Propagation	User action (file execution)	Automatic via network
Self-replication	Inserts into other programs	Independent replication
Primary Damage	File corruption, system failure	Network paralysis, resource exhaustion

Part 3

System and Network Threats

Attacking Network Traffic

Attack Type	Description	Classification
Sniffing	Eavesdropping on network packets to steal information	Passive attack
Spoofing	Forging IP/MAC addresses to impersonate a trusted source	Active attack
Man-in-the-Middle	Intercepting and modifying data during communication	Active attack

Denial of Service (DoS / DDoS)

DoS: Disrupting normal service by sending excessive requests to a server

Type	Description
Resource Exhaustion	Depleting CPU, memory, bandwidth, etc.
Network Disruption	Disabling the network itself
DDoS	Simultaneous attack from multiple zombies (botnet)

SYN Flood Attack:

```
Attacker → [SYN] → Server     (connection request)
Server   → [SYN-ACK] → ???    (waiting for response)
※ ACK never arrives, half-open connections accumulate
→ Server's connection table exhausted → legitimate connections denied
```

- DDoS is sometimes combined with **blackmail** (extortion)

Port Scanning

Port Scanning: Probing open ports to discover vulnerable services

- Not an attack itself, but a **reconnaissance** phase
- Security administrators also use it for service auditing

```
Attacker → Port 21 (FTP)     → No response (closed)
Attacker → Port 22 (SSH)     → Response (open!) → Check version
Attacker → Port 80 (HTTP)    → Response (open!) → Identify web server
Attacker → Port 443 (HTTPS)  → Response (open!)
 ...
```

Fingerprinting: Identifying the OS type and service versions to search for known vulnerabilities

Key tools:

- **nmap**: Network exploration and security auditing tool
- **Metasploit**: Vulnerability exploit testing framework

Part 4

Cryptography

Cryptography Overview

Cryptography: Technology that enables secure communication over untrusted channels in network environments

Components of a cryptographic algorithm:

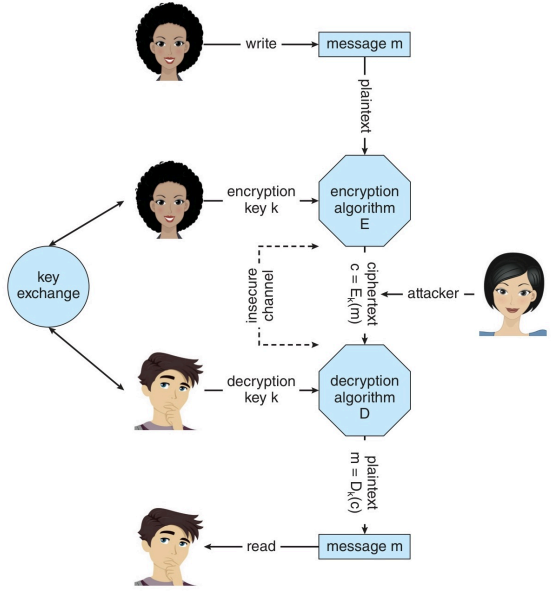
- **K** = Set of keys
- **M** = Set of plaintext messages
- **C** = Set of ciphertexts
- **E** : K → (M → C) – Encryption function
- **D** : K → (C → M) – Decryption function

Key property: Given a ciphertext c, recovering the original message m without the key k is **computationally infeasible**

Type	Key Structure	Speed	Key Exchange
Symmetric	Same key	Fast	Difficult
Asymmetric	Public/private key pair	Slow	Secure

Symmetric Encryption

Encryption and decryption performed with the same key (k)



Silberschatz, Figure 16.7 – A secure communication over an insecure medium

Algorithm	Key Length	Characteristics

Asymmetric Encryption

Encrypt with public key, decrypt with private key

```
Plaintext m ──[Public Key ke]──→ Ciphertext c ──[Private Key kd]──→ Plaintext m
               Encryption                          Decryption
```

RSA Algorithm (Rivest, Shamir, Adleman):

- p, q: Two large prime numbers, N = p × q
- Public key ke: ke × kd mod (p-1)(q-1) = 1
- Encryption: c = m^ke mod N
- Decryption: m = c^kd mod N

RSA Example (small numbers):

```
p=7, q=13 → N=91, (p-1)(q-1)=72
ke=5, kd=29 (5×29 mod 72 = 1)
Encryption: 69^5 mod 91 = 62
Decryption: 62^29 mod 91 = 69
```

Symmetric vs Asymmetric Comparison

Property	Symmetric	Asymmetric
Key Structure	One shared key	Public key + private key pair
Speed	**Fast**	Slow (tens to hundreds of times slower)
Key Exchange	Requires a secure channel	Public key freely distributed
Use Cases	Bulk data encryption	Authentication, key exchange, small data
Representative Algorithms	AES, DES	RSA, ECC

Practical Usage (Hybrid Encryption):

```
1. Exchange symmetric key securely using asymmetric encryption
2. Subsequent communication uses symmetric encryption (better performance)
```

Hashing (Hash Functions)

Hash Function H(m): Arbitrary-length message → **Fixed-length hash value** (message digest)

Properties:

- **One-way**: Cannot recover the original from the hash value
- **Collision Resistant**: Computationally infeasible for different inputs to produce the same hash
- Even a 1-bit change in input **completely changes** the hash (avalanche effect)

```
"Hello World"  ──[SHA-256]──→ a591a6d40bf420404a011733cfb7b190 ...
"Hello World!" ──[SHA-256]──→ 7f83b1657ff1fc53b92dc18148a1d65d ...
```

Algorithm	Output Length	Status
MD5	128-bit	**Not secure**
SHA-1	160-bit	Being phased out
SHA-256	256-bit	**Current standard**

Hash Applications – Password Storage

Store hash values instead of **plaintext passwords**

```
Registration: password → H(password) → Store hash in DB
Authentication: input → H(input) → Compare with stored hash
```

Salt: A random value added to the hash for enhanced security

```
Without salt:  H("password123") → abc123 ...  (same password = same hash)
With salt:     H("password123" + "x7k9")  → def456 ...
               H("password123" + "m2p5")  → ghi789 ...
               (same password with different salts produces different hashes)
```

- **Dictionary Attack Prevention**: With salt, every dictionary word must be hashed for each salt combination
- UNIX `/etc/shadow` file: Stores hashed passwords readable only by the superuser

Digital Signature

A mechanism for verifying the **integrity** and **origin** of a message

- Uses public key cryptography in **reverse**: sign with private key, verify with public key

```
Signature creation:  signature = H(m)^ks mod N   (sign with private key ks)
Signature verification:  H(m) =? signature^kv mod N  (verify with public key kv)
```

MAC (Message Authentication Code):

- Authentication using a symmetric key → only key holders can create/verify

Applications of Digital Signatures:

- **Nonrepudiation**: The signer cannot deny the action
- **Code Signing**: Verifying the integrity and origin of programs
- **Electronic Document Signing**: Digital contracts, certificates

Digital Certificates

An electronic document in which a **Certificate Authority (CA)** vouches for the owner of a public key

```
Certificate:
  Subject: www.example.com
  Public Key: [RSA 2048-bit key]
  Issuer: Trusted CA
  Validity: 2025-01-01 ~ 2026-12-31
  Signature: [Signed with CA's private key]
```

Certificate Chain (Chain of Trust):

```
Root CA (built into browser)
  ↓ signs
Intermediate CA
  ↓ signs
Server Certificate (www.example.com)
```

- **X.509**: Standard format for digital certificates
- The CA's public key is **pre-installed in the browser**
- Man-in-the-Middle prevention: Verify public key authenticity via certificates

TLS (Transport Layer Security)

The encryption protocol underlying HTTPS (successor to SSL)

TLS Handshake Process:

```
Client                              Server
  │                                    │
  │──── ClientHello (nc) ──────────→   │  (1) Send random value
  │                                    │
  │←──── ServerHello (ns) ─────────    │  (2) Server random value +
  │←──── Certificate (certs) ────      │      send certificate
  │                                    │
  │   [Verify certificate]             │  (3) Verify CA signature
  │                                    │
  │──── Eke(pms) ──────────────→       │  (4) Encrypt premaster secret
  │                                    │      with public key and send
  │                                    │
  │   ms = H(nc, ns, pms)              │  (5) Both sides compute
  │   (Generate symmetric session key) │      the same master secret
  │                                    │
  │←──── [AES encrypted comm] ────→    │  (6) Communicate with symmetric key
```

Key Distribution Problem

Symmetric encryption key exchange problem:

- N parties communicating with each other require N(N-1)/2 keys
- A secure channel is needed to deliver keys → out-of-band exchange has low scalability

Asymmetric encryption public key distribution problem:

```
Man-in-the-Middle on Public Key:

Alice ──→ [Request Public Key] ──→ Bob
                    ↑
            Mallory sends her
            own Public Key

Alice encrypts with Mallory's public key
→ Mallory decrypts, then re-encrypts with Bob's public key and forwards
```

Solution: Public key authentication via Digital Certificates

- CA vouches for the public key owner → prevents MITM

Part 5

User Authentication

User Authentication Methods

Three factors for verifying a user's identity:

Factor	Category	Examples
Something you know	Knowledge	Password, PIN, security questions
Something you have	Possession	Smart card, OTP token, smartphone
Something you are	Biometrics	Fingerprint, iris, facial recognition, voice

Password-based Authentication:

- Most common but has various vulnerabilities
- Trade-off between convenience and security

Password Vulnerabilities

Attack Type	Description
Dictionary Attack	Tries dictionary words and variations in sequence
Brute Force	Tries all possible combinations (short passwords are vulnerable)
Social Engineering	Tricks users into directly revealing passwords
Shoulder Surfing	Observing input over someone's shoulder
Sniffing	Stealing passwords from network traffic
Phishing	Inducing password entry via fake websites/emails
Keylogger	Malware that records keyboard input

One-Time Password (OTP)

OTP: A password that can be used only once → prevents sniffing and replay attacks

```
Challenge-Response method:
1. Server → Client: send challenge (ch)
2. Client: compute H(password, ch) → send authenticator
3. Server: perform the same computation → compare results
→ Different challenge each time, so authenticator is different each time
```

OTP Implementation Methods:

- Hardware tokens (RSA SecurID, etc.)
- Software apps (Google Authenticator, etc.)
- SMS authentication codes

Two-Factor Authentication (2FA):

- "Something you have" (token) + "Something you know" (PIN)
- Even if one is compromised, the other provides protection

Multifactor Authentication (MFA)

Combining two or more authentication factors for enhanced security

```
Single factor:  Password only → Vulnerable

2FA examples:   Password (knowledge) + OTP (possession)
                Password (knowledge) + Fingerprint (biometrics)

3FA example:    Password (knowledge) + Smart card (possession) + Facial recognition (biometrics)
```

Auth Level	Factors	Security Strength	Use Cases
Single factor	1	Low	General websites
2FA	2	Medium	Online banking
3FA	3	High	Military/government systems

Biometrics Pros and Cons:

Part 6

Goals and Principles of Protection

Goals of Protection

Protection: OS mechanisms that control resource access by processes and users

Why Protection is needed:

1. Prevent malicious access violations
2. Ensure system resources are used only according to defined policies
3. **Early detection of interface errors** between components → improved system reliability

Policy vs Mechanism:

Aspect	Description	Example
Policy	Decides **what** to do	"Students can only read their own files"
Mechanism	Decides **how** to do it	Access matrix, ACL

- **Separation** of policy and mechanism is important → no need to modify the mechanism when the policy changes

Key Principles of Protection

Principle of Least Privilege:

- Grant users/processes only the **minimum privileges** needed for their tasks
- The practice of not running as root in UNIX is a representative example
- Excessive privileges expand the scope of damage from errors or attacks

Compartmentalization (isolation):

- Divide the system into independent zones
- Prevent a breach in one zone from affecting others
- Examples: DMZ, virtualization, containers

Defense in Depth (layered defense):

- Deploy multiple layers of security → if one layer is breached, the next layer defends

```
[Firewall] → [IDS/IPS] → [OS Access Control] → [Application Auth] → [Data Encryption]
```

- **Audit Trail**: Record access logs for attack detection and analysis

Need-to-Know Principle

Need-to-Know: A process should only be able to access resources needed for its current task

```
When Process P calls Procedure A():
- A() can only access its local variables and passed parameters
- Cannot access other variables of Process P

When Process P invokes the Compiler:
- The Compiler can only access related files (source file, output file, etc.)
- Cannot access arbitrary files
```

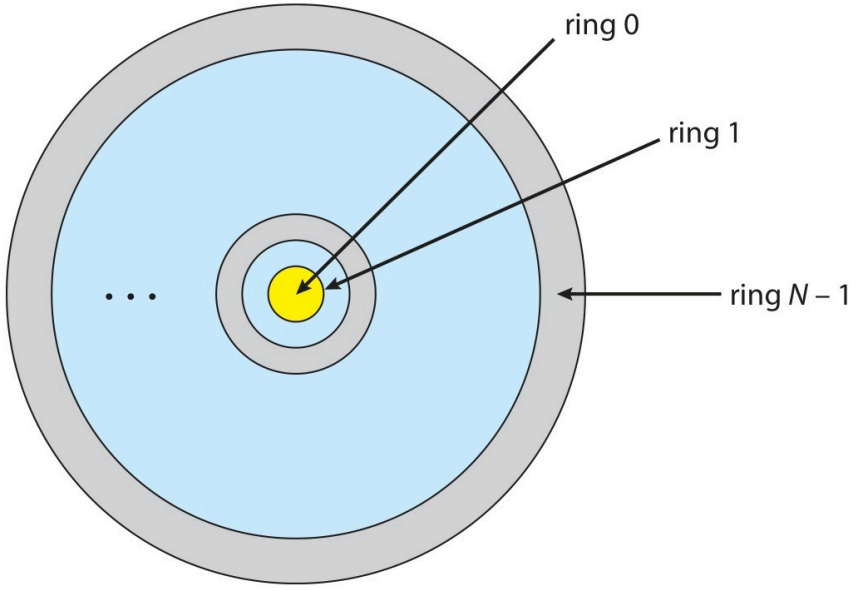
Concept	Role
Need-to-Know	**Policy** – Decides what access to allow
Least Privilege	**Mechanism** – Means to implement the policy

Part 7

Protection Rings

Protection Rings Structure

Hardware-level privilege hierarchy – based on the **Bell-LaPadula model**



Silberschatz, Figure 17.1 – Protection-ring structure

- Ring i provides only a subset of Ring j (j < i) functionality
- Ring 0 has the highest privilege (full privileges)

Intel and ARM Protection Rings

Platform	Structure	Description
Intel x86	Ring 0~3	Mostly only Ring 0 (kernel) and Ring 3 (user) are used
Intel VT-x	Ring -1 added	For hypervisor (VMM)
ARM (early)	USR / SVC	User mode / Supervisor mode
ARM TrustZone	Secure / Normal World	Hardware-based secure zone separation
ARMv8	EL0~EL3	4-level Exception Levels

ARMv8 Exception Levels:

Part 8

Domain of Protection

Domain Structure

Domain: The set of resources and privileges that a process can access

```
Domain = { <object-name, rights-set>, ... }

D1 = { <File1, {read}>, <File2, {read, write}> }
D2 = { <File2, {read}>, <Printer, {write}> }
D3 = { <File1, {execute}>, <File3, {read}> }
```

Domain Association Methods:

- **Static**: Domain fixed for the process lifetime → may grant excessive privileges
- **Dynamic**: Process can switch between domains → adheres to the need-to-know principle

Domain Implementations:

- **User-based**: Per-user domain, switches on logout/login
- **Process-based**: Per-process domain, switches via message passing
- **Procedure-based**: Per-procedure domain, switches on function calls

Domain Switching

A process transitioning from one domain to another

```
User Process (Ring 3, User Domain)
      │
      │ system call (syscall instruction)
      ↓
Kernel (Ring 0, Kernel Domain)    ← domain switch
      │
      │ return
      ↓
User Process (Ring 3, User Domain)
```

UNIX setuid Mechanism:

- Setting the setuid bit on an executable → runs with the **file owner's privileges**
- Example: `passwd` command → owned by root, setuid set → regular users can modify `/etc/shadow`
- Risk: If a setuid binary has a vulnerability → **privilege escalation** is possible

Android Application ID:

- Each app is assigned a unique UID/GID → isolation between apps

Part 9

Access Matrix

Access Matrix Overview

A model that represents resource access rights as a **matrix**

	F1	F2	F3	Printer
D1	read	read, write		
D2		read	read	write
D3	read, write		read, write	
D4	read, write		read, write	

- **Row**: Domain (subject) – users, processes, roles
- **Column**: Object – files, devices, memory regions
- **Cell**: Access Rights – read, write, execute, etc.

Problem: In real systems, most of the matrix is **empty** (sparse matrix) → Efficient implementation methods are needed

Access Matrix Extension – Domain Switching

Include domains as **objects** to control domain switching

	F1	F2	F3	Printer	D1	D2	D3	D4
D1	read		read			switch		
D2		read, write					switch	switch
D3			execute					
D4	read, write		read, write		switch			

- `switch` in access(i, j) → switching from domain Di to Dj is allowed
- Example: D2 can switch to D3 or D4

Modifying Access Rights – Copy, Owner, Control

Copy Right (`*`):

- `read*` → the right can be copied to another domain in the same column
- Transfer: Copy then remove the original
- Limited Copy: Copy without `*` → re-copying not allowed

Owner Right:

- If access(i, j) includes owner → Di can add/delete all entries in column j
- The file owner sets other users' access rights

Control Right:

- Applies only to domain objects
- If access(i, j) includes control → Di can delete access rights in row j

Confinement Problem: The problem of ensuring information does not leak outside the execution environment → Generally unsolvable (undecidable)

Part 10

Implementation of Access Matrix

Global Table

The simplest implementation: A complete table of `<domain, object, rights-set>` triples

```
< D1, F1, {read} >
< D1, F2, {read, write} >
< D2, F2, {read} >
< D2, Printer, {write} >
< D3, F1, {read, write} >
< D3, F3, {read, write} >
...
```

Advantages: Simple to implement

Disadvantages:

- Table is very large, **difficult to store in main memory** → additional I/O required
- Cannot leverage special grouping (e.g., "all domains can read F1" → requires individual entries for every domain)

Access List for Objects (ACL)

Implemented based on **columns** of the Access Matrix – each object has a linked (domain, rights) list

```
File1:    [(D1, {read}), (D3, {read, write})]
File2:    [(D1, {read, write}), (D2, {read})]
File3:    [(D2, {read}), (D3, {read, write})]
Printer:  [(D2, {write})]
```

- **Default Rights**: Can set default access rights for domains not in the list
- On access request: Search the object's ACL to check the domain's rights

Advantages: Directly addresses user needs (specify access rights when creating files)

Disadvantages:

- Difficult to check all access rights for a specific domain
- ACL search required on every access → long lists cause performance degradation

Capability List for Domains

Implemented based on **rows** of the Access Matrix – each domain has a linked (object, rights) list

```
D1:  [(File1, {read}), (File2, {read, write})]
D2:  [(File2, {read}), (File3, {read}), (Printer, {write})]
D3:  [(File1, {read, write}), (File3, {read, write})]
```

Capability: A protected pointer representing access rights to an object

- Possessing a capability grants access
- Managed by the OS; users cannot modify them directly

Methods for Protecting Capabilities:

- **Tag method**: Hardware uses a tag bit to distinguish capabilities from regular data
- **Separate address space**: Store capability lists in separate memory accessible only by the OS

Lock-Key Mechanism

A **compromise** between ACL and Capability List

```
Assign a Lock (bit pattern) to each Object:
  File1: Lock = [1010]
  File2: Lock = [0110]

Assign a Key (bit pattern) to each Domain:
  D1: Key = [1010]  → Matches File1's Lock → Access granted
  D2: Key = [0110]  → Matches File2's Lock → Access granted
```

- Access is granted if the Key matches the Lock
- Keys can be freely transferred between domains
- Changing the Lock enables effective **revocation of rights**

Implementation Method Comparison

Method	Basis	Advantages	Disadvantages
Global Table	Entire matrix	Simple implementation	Large table size, no grouping
ACL	Column (Object)	Intuitive for users, easy revocation	Hard to determine per-domain rights
Capability	Row (Domain)	Fast access verification, process-centric	Difficult revocation
Lock-Key	Matching	Flexible, easy revocation	Key management complexity

Real Systems: Use a **combination** of ACL + Capability

- **File open**: Verify access rights via ACL
- Create a **capability (file descriptor)** → subsequent access verified quickly via capability
- **File close**: Delete the capability

Revocation of Access Rights

Considerations when revoking rights:

Question	Options	
Timing	**Immediate** vs **Delayed**	
Scope	**Selective** vs **General**	
Extent	**Partial** vs **Total**	
Duration	**Temporary** vs **Permanent**	
Method	ACL	Capability
Ease of Revocation	**Easy** – delete from the list	**Difficult** – must find distributed capabilities

Part 11

Access Control Models

RBAC (Role-Based Access Control)

Access control through User → **Role** → Permission mapping

```
Users            Roles              Permissions

 Alice  ──→      Admin    ──→       File: read/write
  Bob   ──→      Editor   ──→       File: read/write
 Carol  ──→      Viewer   ──→       File: read
```

Advantages of RBAC:

- Efficient management at the **role level** even with many users
- Easy to apply the **principle of least privilege** – assign only necessary roles
- When a user leaves the organization, simply remove them from the role
- First fully adopted in Solaris 10
- Assign privileges to roles, then assign roles to users

DAC vs MAC

Property	DAC (Discretionary)	MAC (Mandatory)
Definition	Resource owner sets access rights	System-enforced access control policy
Permission Setting	User (owner)	System administrator/policy
Root Privileges	Root can access everything	**Root cannot change the policy**
Flexibility	High	Low (subject to policy)
Security Strength	Relatively low	High
Examples	UNIX file permissions	SELinux, macOS SIP

Limitations of DAC:

MAC – Security Labels

Core of MAC: **Labels** (security labels)

- Assign labels to subjects (processes) and objects (files, devices)
- Policy determines whether access between labels is allowed/denied

```
Security levels:  Unclassified < Secret < Top Secret

"Secret" user:
  ✓ Can access "Unclassified" files
  ✓ Can access "Secret" files
  ✗ Cannot access "Top Secret" files (cannot even see they exist)
```

SELinux (Security-Enhanced Linux):

- MAC implementation in the Linux kernel, developed by the NSA
- Fine-grained access control via policy files
- Integrated into most Linux distributions

Other implementations: macOS (TrustedBSD-based), Windows (Mandatory Integrity Control)

Linux Capabilities

Subdividing root privileges to grant processes only the **necessary permissions**

```
Traditional UNIX:  root = all privileges (all-or-nothing)

Linux Capabilities:
  CAP_NET_BIND_SERVICE  → Bind to ports below 1024
  CAP_SYS_ADMIN         → System administration tasks
  CAP_DAC_OVERRIDE      → Override file access controls
  CAP_NET_RAW           → Use raw sockets
   ... (managed as bitmasks)
```

Three Bitmasks:

- **Permitted**: Capabilities that are allowed
- **Effective**: Currently active capabilities
- **Inheritable**: Capabilities that can be inherited by child processes
- Once a capability is **revoked, it cannot be reacquired**
- Direct implementation of the **principle of least privilege**

Part 12

Sandboxing and Code Signing

Sandboxing

Isolating a program's execution environment to limit its impact on the system

Example	Description
Web Browser	Isolates each tab/plugin in a separate process
Mobile App	Independent filesystem and permission system per app
Container	Application environment isolation via Docker, etc.
Virtual Machine	Complete isolation at the hardware level
Java/JVM	Sandbox restrictions at the virtual machine level

Sandboxing Implementation Methods:

Sandboxing Profile Example

macOS Sandbox Profile (Scheme language):

```
(version 1)
(deny default)                              ; Deny all operations by default
(allow file-chroot)                         ; Allow chroot
(allow file-read-metadata (literal "/var")) ; Allow reading /var metadata
(allow sysctl-read)                         ; Allow sysctl reads
(allow mach-per-user-lookup)
(allow mach-lookup
  (global-name "com.apple.system.logger"))  ; Allow access only to logger service
```

SECCOMP-BPF (Linux):

- Define system call filters using Berkeley Packet Filter language
- Apply to processes via `prctl()` system call
- Restrictions inherited by child processes on fork
- Automatically applied to all apps in Android Bionic C library

Code Signing

Verify code **integrity** and **origin** through **digital signatures**

```
Developer                          User System
                                   ┌──────────┐
 ┌──────┐
 │ Code │ ──[Private Key]──→ Signature
 └──────┘                          │
                                   │ Verification
Code + Signature ──[Public Key]──→ Integrity/origin check
                                   │
                                   ├─ Success: Allow execution
                                   └─ Failure: Block execution or warn
```

OS	Implementation
Windows	Driver Signing, Authenticode
macOS	Gatekeeper, SIP (System Integrity Protection)
iOS	All apps require signing, Apple signs for App Store distribution

System Integrity Protection (SIP)

A **system protection mechanism** introduced in macOS 10.11

SIP Features:

- Restricts access to system files and resources
- **Even the root user** cannot modify system files
- Only allows code-signed kernel extensions
- Prevents debugging/tampering of system binaries

```
Traditional UNIX:
  root → Can access/modify all files

With SIP:
  root → Can manage other users' files
         Can install/remove programs
         ✗ Cannot modify system files
         ✗ Cannot tamper with kernel extensions
```

- SIP is enforced on all processes at boot time
- Only Apple-signed binaries with **Entitlements** are exceptions

Part 13

Lab – Implementing an Access Control System

Lab – Access Matrix Implementation

Access Matrix-based permission checking system

```
class AccessMatrix:
    def __init__(self):
        self.matrix = {}  # {domain: {object: set(rights)}}

    def add_right(self, domain, obj, right):
        if domain not in self.matrix:
            self.matrix[domain] = {}
        if obj not in self.matrix[domain]:
            self.matrix[domain][obj] = set()
        self.matrix[domain][obj].add(right)

    def check_access(self, domain, obj, right):
        if domain in self.matrix:
            if obj in self.matrix[domain]:
                return right in self.matrix[domain][obj]
        return False

    def revoke_right(self, domain, obj, right):
        if domain in self.matrix:
            if obj in self.matrix[domain]:
                self.matrix[domain][obj].discard(right)
```

Lab – ACL and Capability List Implementation

```python
class ACL:
    """Column (Object) based – (domain, rights) list per object"""
    def __init__(self):
        self.acl = {}  # {object: {domain: set(rights)}}

    def grant(self, obj, domain, right):
        self.acl.setdefault(obj, {}).setdefault(domain, set()).add(right)

    def check(self, obj, domain, right):
        return right in self.acl.get(obj, {}).get(domain, set())

    def revoke(self, obj, domain, right):
        if obj in self.acl and domain in self.acl[obj]:
            self.acl[obj][domain].discard(right)


class CapabilityList:
    """Row (Domain) based – (object, rights) list per domain"""
    def __init__(self):
        self.caps = {}  # {domain: {object: set(rights)}}

    def grant(self, domain, obj, right):
        self.caps.setdefault(domain, {}).setdefault(obj, set()).add(right)
```

Lab – RBAC Implementation

```python
class RBAC:
    def __init__(self):
        self.roles = {}          # {role: {object: set(rights)}}
        self.user_roles = {}     # {user: set(roles)}

    def add_role(self, role):
        if role not in self.roles:
            self.roles[role] = {}

    def assign_permission(self, role, obj, right):
        if role not in self.roles:
            self.roles[role] = {}
        if obj not in self.roles[role]:
            self.roles[role][obj] = set()
        self.roles[role][obj].add(right)

    def assign_role(self, user, role):
        if user not in self.user_roles:
            self.user_roles[user] = set()
        self.user_roles[user].add(role)

    def check_access(self, user, obj, right):
        if user not in self.user_roles:
            return False
```

Lab – Simulation

```python
# Access Matrix test
am = AccessMatrix()
am.add_right("D1", "File1", "read")
am.add_right("D1", "File2", "read")
am.add_right("D1", "File2", "write")
am.add_right("D2", "File2", "read")
am.add_right("D2", "Printer", "write")

print(am.check_access("D1", "File2", "write"))   # True
print(am.check_access("D2", "File2", "write"))   # False

# RBAC test
rbac = RBAC()
rbac.add_role("admin")
rbac.add_role("editor")
rbac.add_role("viewer")
rbac.assign_permission("admin", "File1", "read")
rbac.assign_permission("admin", "File1", "write")
rbac.assign_permission("viewer", "File1", "read")
rbac.assign_role("Alice", "admin")
rbac.assign_role("Bob", "viewer")

print(rbac.check_access("Alice", "File1", "write"))  # True
print(rbac.check_access("Bob", "File1", "write"))    # False
```

Summary

Key Concepts This Week

Topic	Key Content
CIA	Confidentiality, Integrity, Availability
Malware	Trojan, Spyware, Ransomware, Back Door
Code Injection	Buffer Overflow, SQL Injection, Shellcode
Virus vs Worm	Host requirement, propagation method differences
Network Threats	Sniffing, Spoofing, DoS/DDoS, MITM, Port Scanning
Cryptography	Symmetric (AES), Asymmetric (RSA), Hashing (SHA)

Semester Summary

Week	Topic	Textbook
1-2	Process Concepts and Management	Ch 3, 4
3	IPC (Inter-Process Communication)	Ch 3
4-5	Threads and Concurrency	Ch 4
6-7	CPU Scheduling	Ch 5
8	Midterm Exam	
9	Synchronization (Mutex, Semaphore, Monitor)	Ch 6, 7
10	Deadlocks	Ch 8

Final Exam Information

Scope:

- Ch 6, 7: Synchronization (Mutex, Semaphore, Monitor, Deadlock-related synchronization)
- Ch 8: Deadlocks (Detection, Avoidance, Prevention)
- Ch 9: Main Memory (Paging, Segmentation)
- Ch 10: Virtual Memory (Demand Paging, Page Replacement)
- Ch 11: Storage Management (HDD/SSD Scheduling, RAID)
- Ch 16, 17: Security and Protection

Exam types:

- Conceptual essay questions
- Algorithm computation problems (Page Replacement, Disk Scheduling, etc.)
- Code analysis (synchronization, access control, etc.)
- Comparison/analysis (ACL vs Capability, Symmetric vs Asymmetric, etc.)